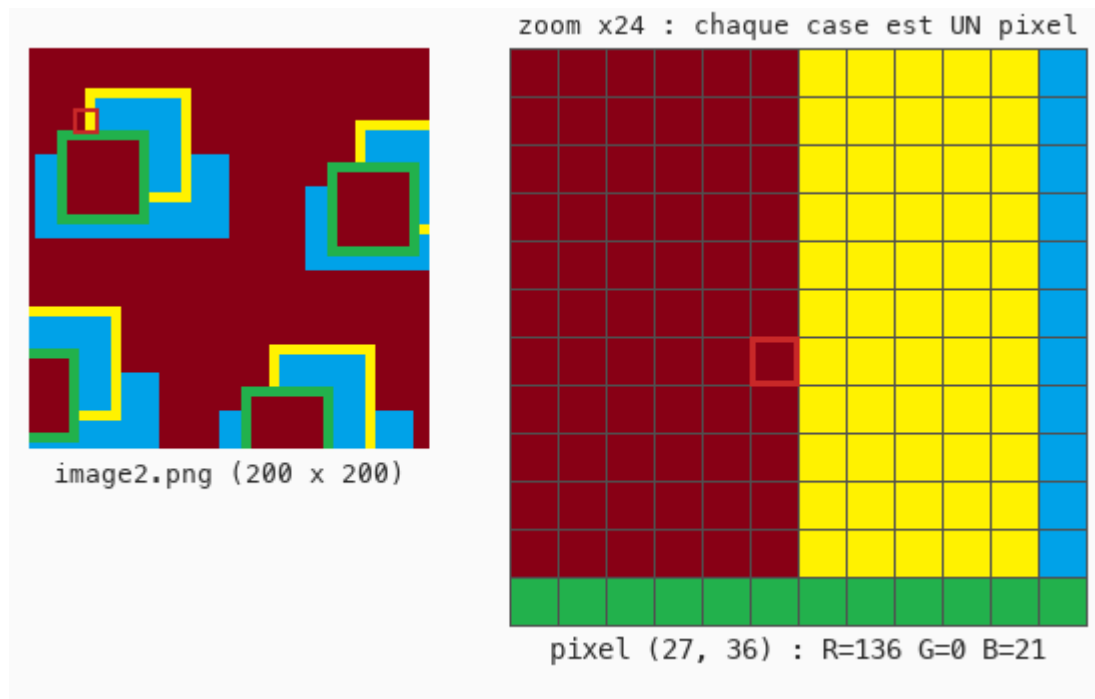


Cours 10 – Image et pixel

Fichiers : ofApp10.h + ofApp10.cpp , image data/image2.png **Avant :** cours 09 (ofColor).

Jusqu'ici tout ce qui était à l'écran venait de nos ordres de dessin. On charge maintenant une image depuis un fichier, on l'affiche, et surtout on **lit** la couleur d'un de ses pixels. C'est la porte d'entrée de tout le traitement d'image des cours suivants.



■ 1. Une image est un tableau de pixels

Un fichier image, une fois chargé, est une grille : largeur × hauteur cases, et dans chaque case une couleur. `image2.png` fait 200 × 200, soit 40 000 pixels. Le pixel en haut à gauche est à (0, 0), celui en bas à droite à (199, 199). Même repère que l'écran (cours 00).

Le zoom ci-dessus montre ce qu'est vraiment un « bord » : il n'y a pas de lignes, seulement des cases de couleurs différentes qui se touchent.

■ 2. Charger une image

```
// dans le .h
ofImage img;
```

```
// dans setup()
bool ok = img.load("image2.png");
if (!ok) {
    ofLogError() << "image2.png introuvable dans bin/data";
}
```

`ofImage` est le type « image ». `img.load(nom)` lit le fichier. Le chemin est **relatif au dossier** `bin/data` du projet : c'est là que doivent aller toutes les images, jamais à côté du `.cpp`.

`load` renvoie un `bool`, un type qui ne vaut que `true` (vrai) ou `false` (faux). Le `!` se lit « non » : `if (!ok)` veut dire « si ça n'a pas marché ». Sans cette vérification, une image introuvable ne provoque aucune erreur visible, juste une fenêtre vide, et on cherche longtemps. Prends l'habitude de vérifier.

`ofLogError()` << ... écrit un message dans la console, comme `std::cout`, mais marqué comme une erreur.

■ 3. Afficher

```
ofSetColor(255);  
img.draw(0, 0);
```

`img.draw(x, y)` affiche l'image avec son coin haut-gauche en `(x, y)`. Le `ofSetColor(255)` juste avant n'est pas décoratif : la couleur courante **teinte** l'image. Si la dernière couleur choisie était rouge, l'image s'affiche rougie. Blanc = image telle quelle.

■ 4. Lire un pixel

```
int px = ofClamp(mouseX, 0, img.getWidth() - 1);  
int py = ofClamp(mouseY, 0, img.getHeight() - 1);  
ofColor c = img.getColor(px, py);
```

`img.getColor(x, y)` renvoie la couleur du pixel `(x, y)`, sous forme d'une `ofColor` (cours 09). On peut alors lire `c.r`, `c.g`, `c.b`.

Le `ofClamp` est indispensable. Lire le pixel `(250, 40)` d'une image de 200 de large, c'est lire hors du tableau : au mieux n'importe quoi, au pire un plantage. `ofClamp(valeur, min, max)` **borne** la valeur : si elle dépasse `max` elle devient `max`, si elle est sous `min` elle devient `min`. Ici la souris peut sortir de l'image, la coordonnée lue, non. On retrouve le problème d'indice hors liste du cours 04 : C++ ne vérifie pas, c'est toi qui bornes.

Remarque `img.getWidth() - 1` : le dernier pixel est à `largeur - 1`, comme le dernier élément d'une liste est à `size() - 1`.

■ 5. Afficher ce qu'on a lu

```
std::string texte = "R=" + ofToString(c.r) + " G=" + ofToString(c.g) + " B=" + ofToString(c.b);  
ofSetColor(255, 200, 0);  
ofDrawBitmapString(texte, 10, 10);  
  
ofSetColor(c);  
ofDrawRectangle(mouseX + 10, mouseY + 10, 20, 20);
```

La concaténation de texte du cours 04, et un petit carré de la couleur lue, à côté du curseur. Passe la souris sur les zones bleues, jaunes, vertes : les nombres changent, le carré aussi. Tu lis l'image comme un tableau de nombres.

■ 6. Un point de vocabulaire : lire l'image, pas l'écran

`getColor` lit dans l'**image chargée**, pas dans ce qui est affiché. Si tu dessines un cercle par-dessus l'image, `getColor` ne le voit pas. C'est un avantage : l'image est une source de données stable, indépendante de ce qu'on dessine. Au cours 12 on lira des pixels d'une image sans même l'afficher.

■ Exercices

1. Charge `pandaroux.jpg` à la place. Adapte la taille de la fenêtre :
`ofSetWindowShape(img.getWidth(), img.getHeight())` après le chargement.
2. Affiche la luminosité du pixel : $(c.r + c.g + c.b) / 3$. Attention à la division entière si tu veux une virgule.
3. Dessine le carré de couleur avec un rayon proportionnel à cette luminosité.
4. Affiche l'image deux fois, côte à côte, la deuxième teintée en rouge par `ofSetColor(255, 100, 100)`.
5. Affiche la teinte du pixel avec `c.getHue()` et dessine à côté un carré de couleur `ofColor::fromHsb(c.getHue(), 255, 255)` : la couleur « pure » correspondante.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp10.cpp`.

Le fichier `ofApp10.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```
#pragma once
#include "ofMain.h"

// 10 - Image et lecture de pixel
// Sketch d'origine : Cours 2022-2023/rendu02
// Notions : ofImage, chargement depuis bin/data, affichage, lecture de la couleur
//           d'un pixel (getColor), affichage de texte, conversion nombre -> texte
// Ressource : bin/data/image2.png

class ofApp : public ofBaseApp {
public:
    void setup();
    void draw();

    ofImage img;
};
```

En tête du fichier `ofApp10.cpp`

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

Étape 1 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```
void ofApp::setup() {
    ofSetWindowShape(200, 200);

    // Le chemin est relatif au dossier bin/data du projet, jamais au fichier .cpp.
    // load() renvoie false si le fichier est introuvable : toujours vérifier.
    bool ok = img.load("image2.png");
    if (!ok) {
        ofLogError() << "image2.png introuvable dans bin/data";
    }
}
```

Étape 2 — `draw()`

Exécuté à chaque frame, après `update()` : uniquement du dessin.

```
void ofApp::draw() {
    ofSetColor(255);          // sans ça, l'image est teintée par la dernière couleur
    img.draw(0, 0);

    // get(mouseX, mouseY) en Processing lit le pixel de l'écran.
    // Ici on lit le pixel de l'IMAGE : il faut rester dans ses limites.
    int px = ofClamp(mouseX, 0, img.getWidth() - 1);
    int py = ofClamp(mouseY, 0, img.getHeight() - 1);
    ofColor c = img.getColor(px, py);

    // red(c) / green(c) / blue(c) => c.r / c.g / c.b
    std::string texte = "R=" + ofToString(c.r) + " G=" + ofToString(c.g) + " B=" + ofToString(c.b);

    ofSetColor(255, 200, 0);
    ofDrawBitmapString(texte, 10, 10);

    // Petit carré de la couleur lue, à côté du curseur
    ofSetColor(c);
    ofDrawRectangle(mouseX + 10, mouseY + 10, 20, 20);
}
```

■ Ce qu'il faut retenir

- Une image est une grille de pixels, chaque pixel une couleur. Le pixel `(x, y)` : `x` de 0 à `largeur - 1`, `y` de 0 à `hauteur - 1`.
- `ofImage img; img.load("f.png")` charge depuis `bin/data`. Vérifier le `bool` renvoyé avec `if (!ok)`.
- `ofSetColor(255); img.draw(x, y);` affiche sans teinter.
- `ofColor c = img.getColor(x, y);` lit un pixel ; `c.r`, `c.g`, `c.b` ses composantes.

- `ofClamp(v, min, max)` borne une valeur. Toujours borner une coordonnée avant de lire un pixel.